Filter

Getting started
- Design principles
- Designing for iOS
- Designing for iPadOS
- Designing for macOS
- Designing for tvOS
- Designing for visionOS
- Designing for watchOS
- Designing for games
- Designing for iPhone Duo

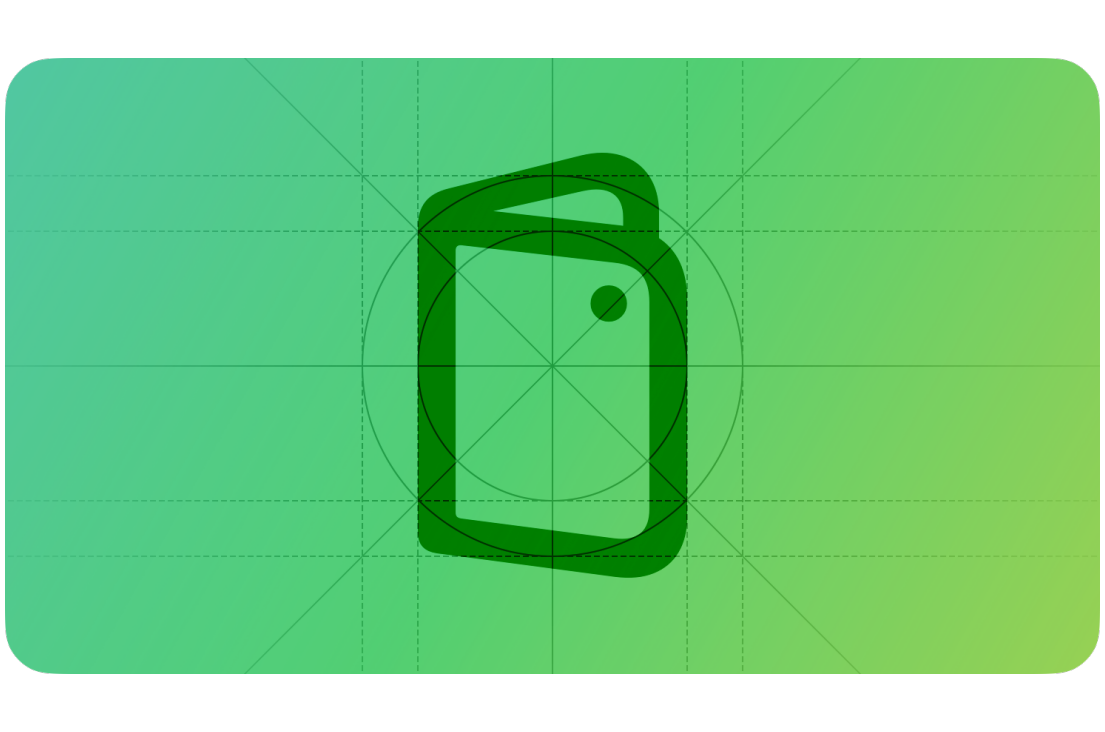
Foundations
Patterns
Components
Inputs
Technologies

September 9, 2026
New page. Introduces the fundamental concepts of designing for iPhone Duo, including device poses, dynamic layouts across dual displays, and toolbars and tab bars on the vertical axis.

Designing for iPhone Duo
Anatomy
Best practices
Dynamic layouts
Vertical controls
Resources
Change log

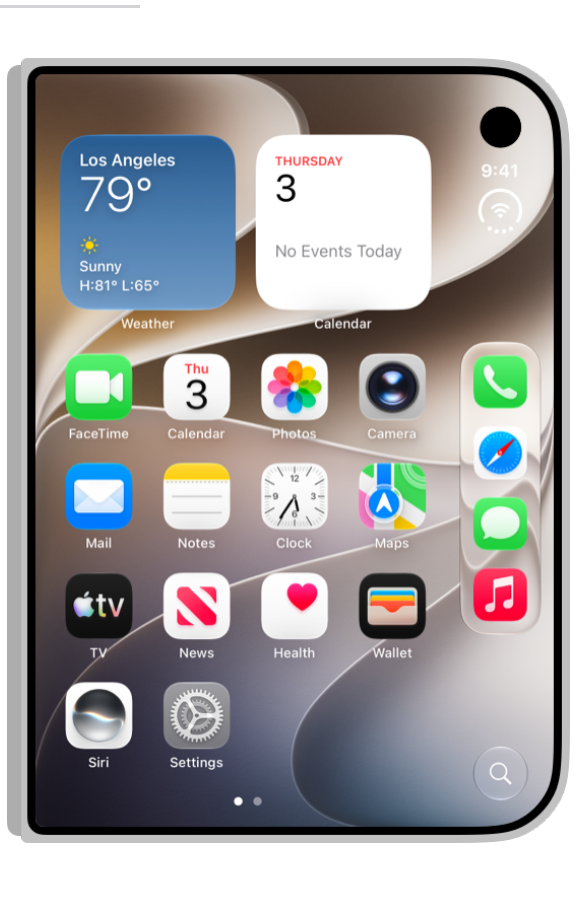
Designing for iPhone Duo

An app designed for iPhone Duo adapts seamlessly to both displays, providing a continuous experience as the device opens and closes.



iPhone Duo has two displays, each with its own front-facing camera. A hinge in the center lets people open and close the device, and supports a variety of ways to hold and position it. This range of display sizes and poses makes an adaptable layout more important than ever. If your app uses standard system components and you've designed it to support resizing, it automatically adapts to the device's poses with little adjustment required. For developer guidance, see Preparing your app for iPhone Duo.

Outer display Inner display



Although iPhone Duo is a new form factor, keep in mind that you're still designing for iPhone, and Designing for iOS patterns and best practices still apply.

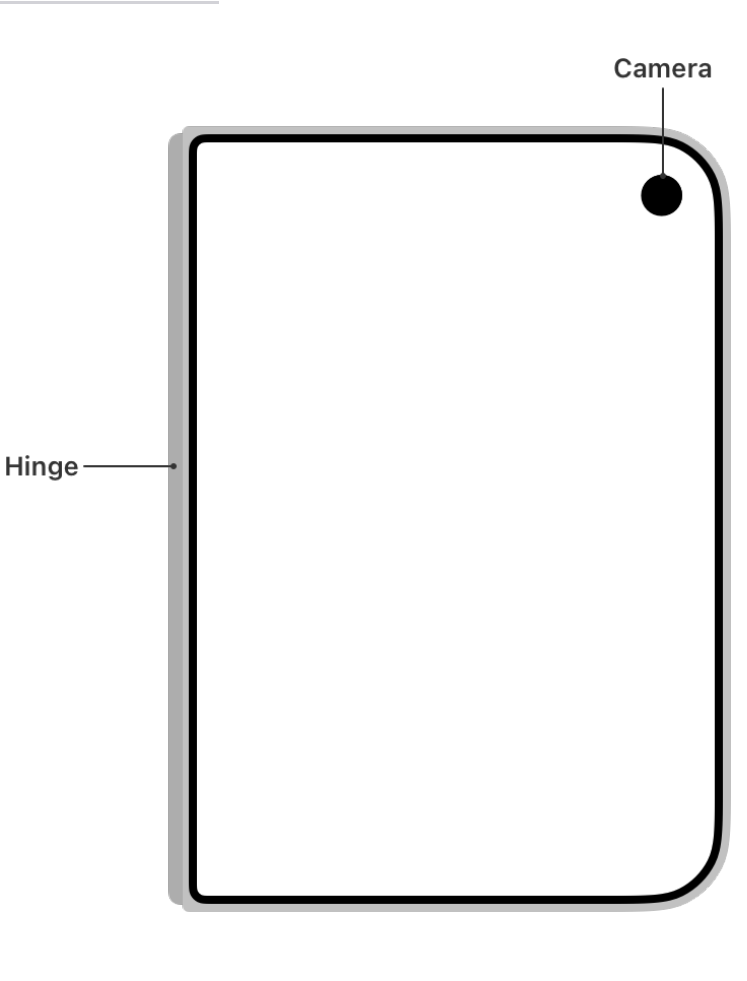
Anatomy

iPhone Duo has an inner and an outer display. People interact with the outer display when the device is closed, and the system places toolbars and tab bars on the side to maximize the vertical space for content. The controls remain on the side when the device opens in landscape to ensure a consistent experience as people move between displays.

A center hinge supports a range of ways to hold and position the device. The hinge also impacts the space available for your content as the device folds.

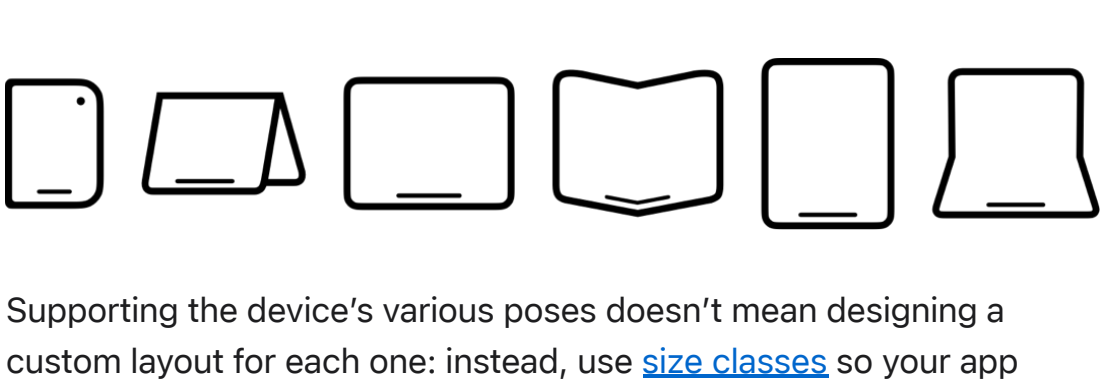
The outer front-facing camera is in the corner and is always visible, vertically aligned with controls on the side. The inner camera is behind the display and stays hidden until the camera is active.

Outer display Inner display



Device poses

People hold iPhone Duo and set it down in a number of ways: partially folded like a book, placed down on a surface, or standing on its edges.



Supporting the device's various poses doesn't mean designing a custom layout for each one: instead, use size classes so your app adapts naturally as it changes size. A compact width layout for the outer display and a regular width layout for the inner display give you the fundamentals for every pose. Don't reinvent your app when it resizes; allow the existing layout to expand based on the available space instead. See Dynamic layouts for guidance.

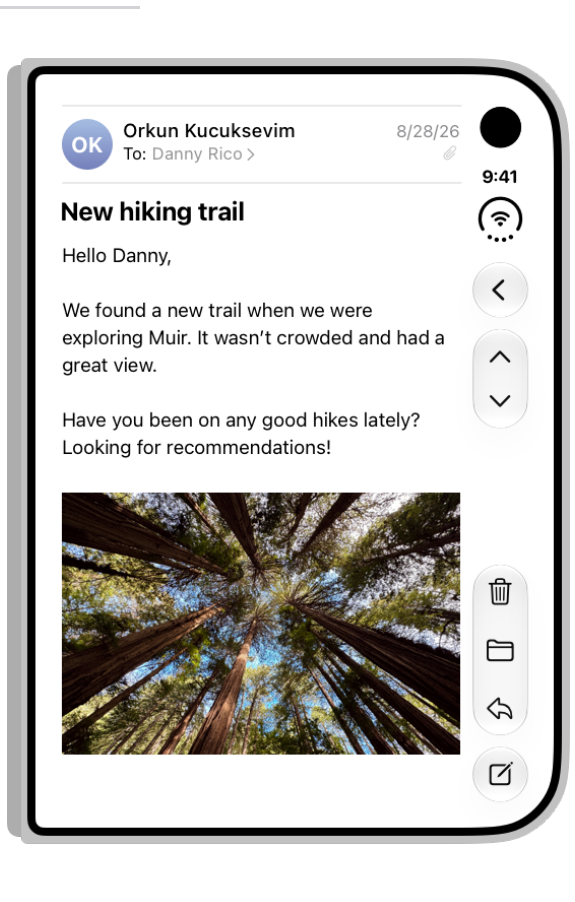
You can use Device Hub in Xcode to preview your app on iPhone Duo and test how your app appears in its various poses.

Best practices

Build your app to resize. Because the device has two displays and supports a wide range of poses and Split View multitasking, your app can appear at many different sizes. Use size classes, layout margins, and safe area insets to lay out controls and content. Avoid fixed widths and display-specific dependencies. See Dynamic layouts below and Layout for guidance.

Create a consistent experience across displays. Keep functionality and the state of elements the same between displays. Maintain your app's information hierarchy, but show an additional level of hierarchy on the larger inner display if it makes sense for your content. Mail, for example, shows either a list of emails (primary) or an email (secondary) when the device is closed. When it's open, it shows both side by side.

Outer display Inner display



Orkun Kucuksevim
To: Danny Rico
8/28/26

New hiking trail

Hello Danny,

We found a new trail when we were exploring Muir. It wasn't crowded and had a great view.

Have you been on any good hikes lately? Looking for recommendations!

Maintain the same functionality across device poses. Controls may overflow and content may move or change size as the interface adapts to the available area. Provide access to the same controls and content regardless of how someone holds or views the device. For guidance, see Dynamic layouts and Vertical controls.

Follow the system's vertical layout for toolbars, tab bars, and navigation controls. Because the outer display is wider and shorter than the display on other iPhone devices, the system moves controls to the side to preserve vertical space for content and reflect the asymmetry of the display. On the inner display, controls remain on the side in landscape to preserve a continuous experience at the same vertical height. If you use standard system components, you receive this layout automatically, but you may want to refine it based on the needs of your app. For guidance, see Vertical controls.

Make your game playable in every device pose. You can choose to lock to either portrait or landscape orientation, but be sure to fill the screen as the device pose changes. When resizing, keep text and control sizes as consistent as possible. Prefer changing the aspect ratio over letterboxing or pillarboxing in games; if you can't avoid letterboxing or pillarboxing, add artwork to the padding area to help the experience feel full screen. See Designing for games for additional guidance.

Dynamic layouts

Designing for iPhone Duo means accounting for a variety of hardware and software configurations. As with all iOS devices, build your layouts with layout margins and safe area insets, and steer clear of fixed widths or anything tied to a specific display.

For guidance, see Layout. For margins and safe areas, see Apple Design Resources. For developer guidance, see `safeAreaInsets` (SwiftUI) and `safeAreaInsets` (UIKit).

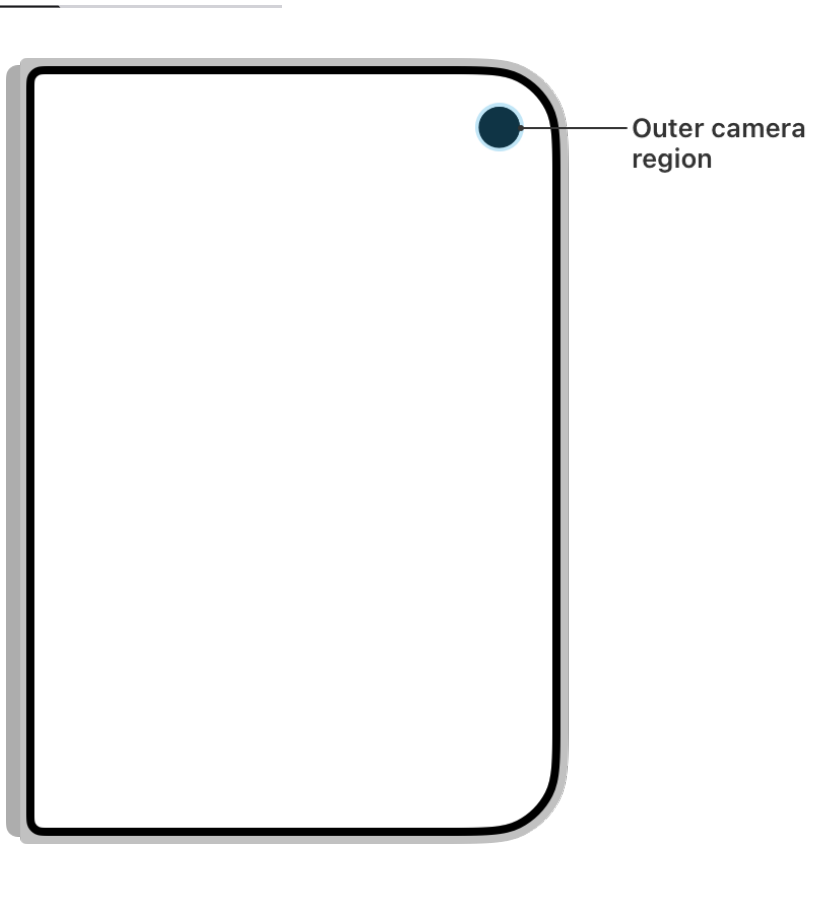
Reserved regions

In addition to standard considerations for safe areas, available space on iPhone Duo is shaped by *reserved regions*. These represent areas within the display that content avoids covering, or that components adapt to accommodate. These are familiar if your layout adapts to similar areas on other platforms, such as the window controls on iPad.

The reserved regions on iPhone Duo include:

- **The outer front-facing camera.** This region is always present, and expands into the Dynamic Island for Live Activities. When controls are on the side, the system automatically accounts for it and arranges elements accordingly.
- **The inner front-facing camera.** This region is only present when the camera is active. When it's inactive, the camera isn't visible; when the camera activates, the UI moves aside to indicate the presence of the camera.
- **The folding region.** This region is conditional based on how a person uses the device. When the device is partially open, the folding region divides the inner display into multiple usable regions, excluding the region at the center as the display folds.

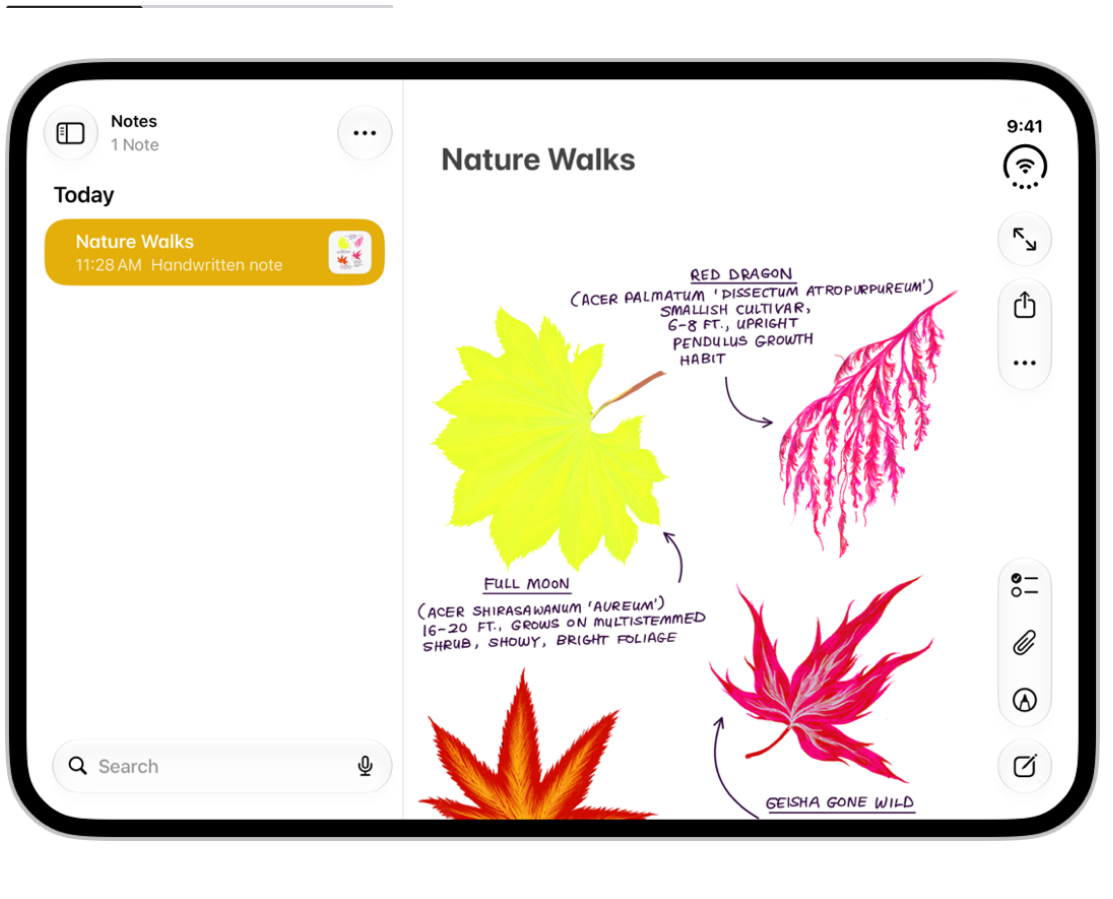
Outer display Inner display



Many system components automatically adapt to reserved regions. Components like alerts, context menus, and sheets automatically move to account for the fold, while larger components like split views adapt their columns' width and margins to match the symmetry of the inner display. For custom components, `ReservedRegion` (SwiftUI) and `UIView.ReservedRegion` (UIKit) provide a way to reposition content away from reserved regions.

Adapt your layout when the device folds. Prefer a layout container that adapts automatically, like the split view in Notes that adjusts the width of each pane to stay clearly visible as the device folds. In a grid-style layout, prefer an even number of columns so content divides cleanly. Use the `ReservedRegion` API to keep important elements clear of the center if the system doesn't move them automatically.

Fully open Partially folded



Avoid extreme layout changes as people fold the device. Move only what's necessary to keep elements visible and easy to tap. Controls that disappear or shift dramatically are harder to find and track, so favor small adjustments over rearrangement.

Split views

On iPhone Duo, a split view expands on the inner display and collapses to a single pane on the outer display, the same way it adapts between regular and compact environments on other iPhone devices. When built with standard components, split views adapt to reserved regions automatically, adjusting width and margins to adapt to the fold.

For general guidance, see split views. For developer guidance, see `NavigationSplitView` (SwiftUI) and `UISplitViewController` (UIKit).

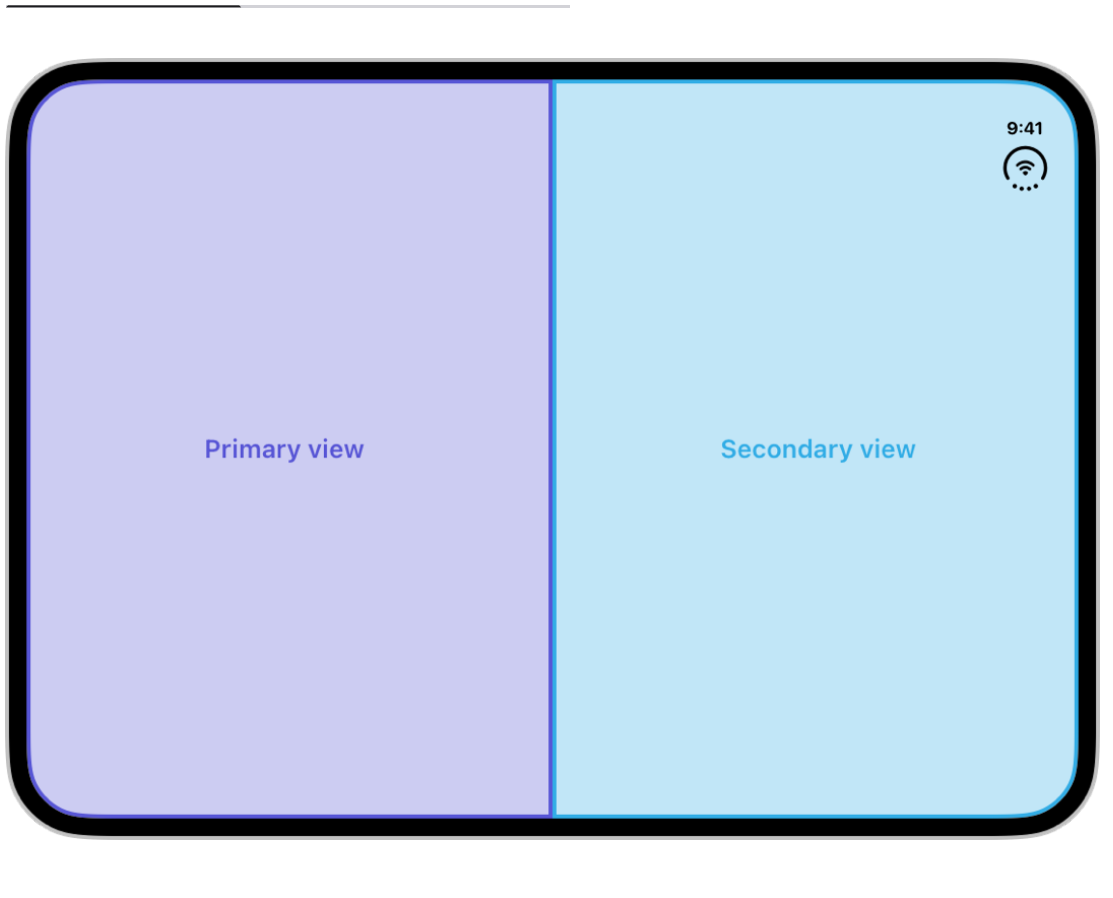
Arrangement views

An *arrangement view* is a layout container that holds two views inside it — a primary view and a secondary view — and dynamically organizes them based on display size, orientation, and reserved regions.

There are two types of arrangement view: split and overlay.

- A *split* arrangement divides its area between its primary and secondary views. It splits horizontally when the arrangement is wider than it is tall, and splits vertically when the arrangement is taller than it is wide.
- An *overlay* arrangement positions the primary and secondary views on top of one another. When the display is partially folded, the views move to occupy each side; otherwise the primary view moves atop the secondary view.

Split arrangement Overlay arrangement



You can limit which axes a split arrangement uses, and collapse the secondary view in an overlay arrangement when you don't want it to appear.

For developer guidance, see `ArrangementView` (SwiftUI) and `UIArrangementViewController` (UIKit).

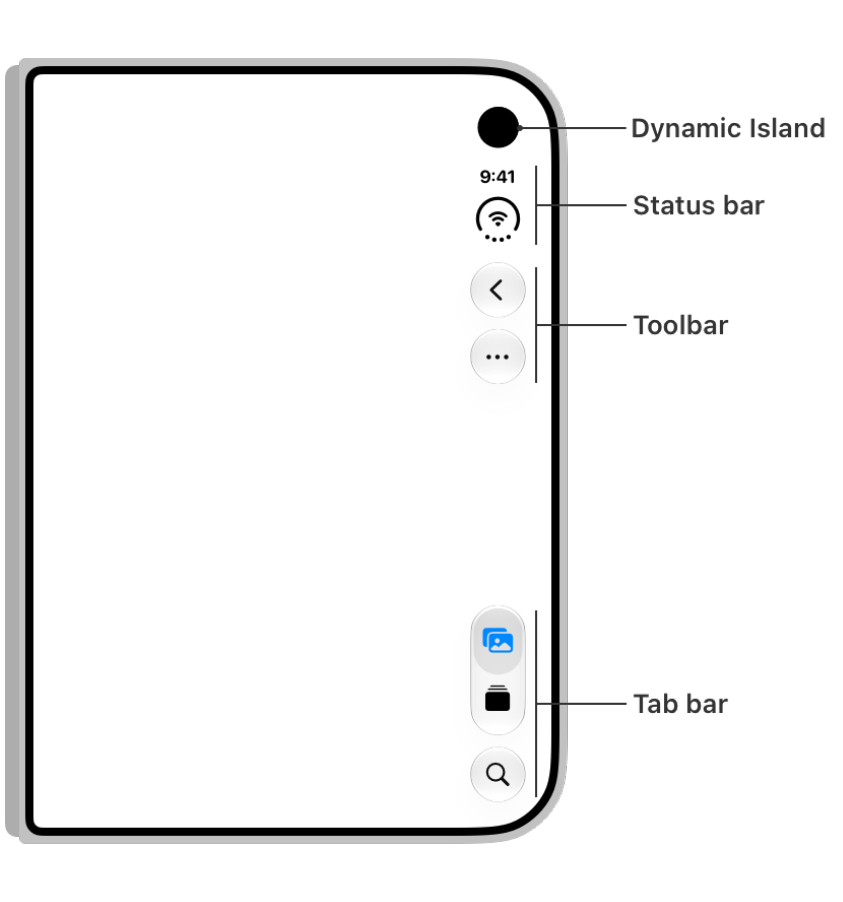
Consider an arrangement view when your layout already resembles one. A layout that places two views side by side or one above the other, such as an `HStack` or `VStack`, translates directly to a split arrangement. A layout that layers one view over another, such as a `ZStack`, translates to an overlay arrangement.

Keep navigation outside of arrangement views. An arrangement view lays out content but doesn't handle navigation, so place navigation containers like navigation split views and tab views around it rather than within it.

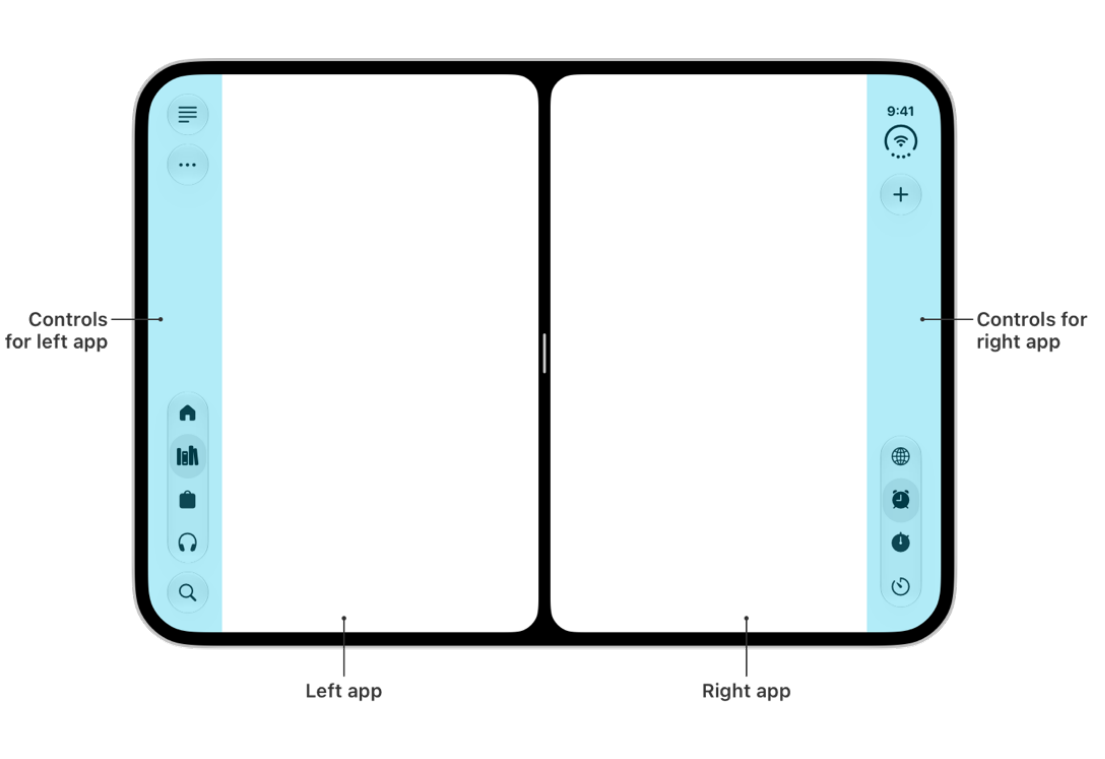
Vertical controls

On iPhone Duo, toolbars, tab bars, and navigation controls that are typically at the top and bottom of the display move to the side, preserving vertical space for content and keeping controls within easy reach. The exception is the inner display in portrait, which has enough vertical space to keep standard horizontal bars.

Controls on the side include both system and app elements: the Dynamic Island, the status bar, the toolbar (including navigation buttons), and the tab bar.



When two apps share the inner display with Split View multitasking, each one places controls along its outer edge, so the left app has controls on the left.



Because controls on the vertical axis stay aligned with the hardware, they hold the same position relative to the camera on the outer display, and stay on the same side in right-to-left languages.

Account for asymmetry in your layouts. Because controls sit along one edge, the space for content is asymmetrical. Use safe areas to make sure controls don't cover your content, including controls on the opposite edge, like when two apps share the inner display with Split View multitasking.

Keep controls consistent across device poses. Because not every pose places controls vertically on the side, and there isn't always the same amount of space available, it's important to keep controls' relative positions as similar as possible so people don't have to relearn where actions live as they change between poses.

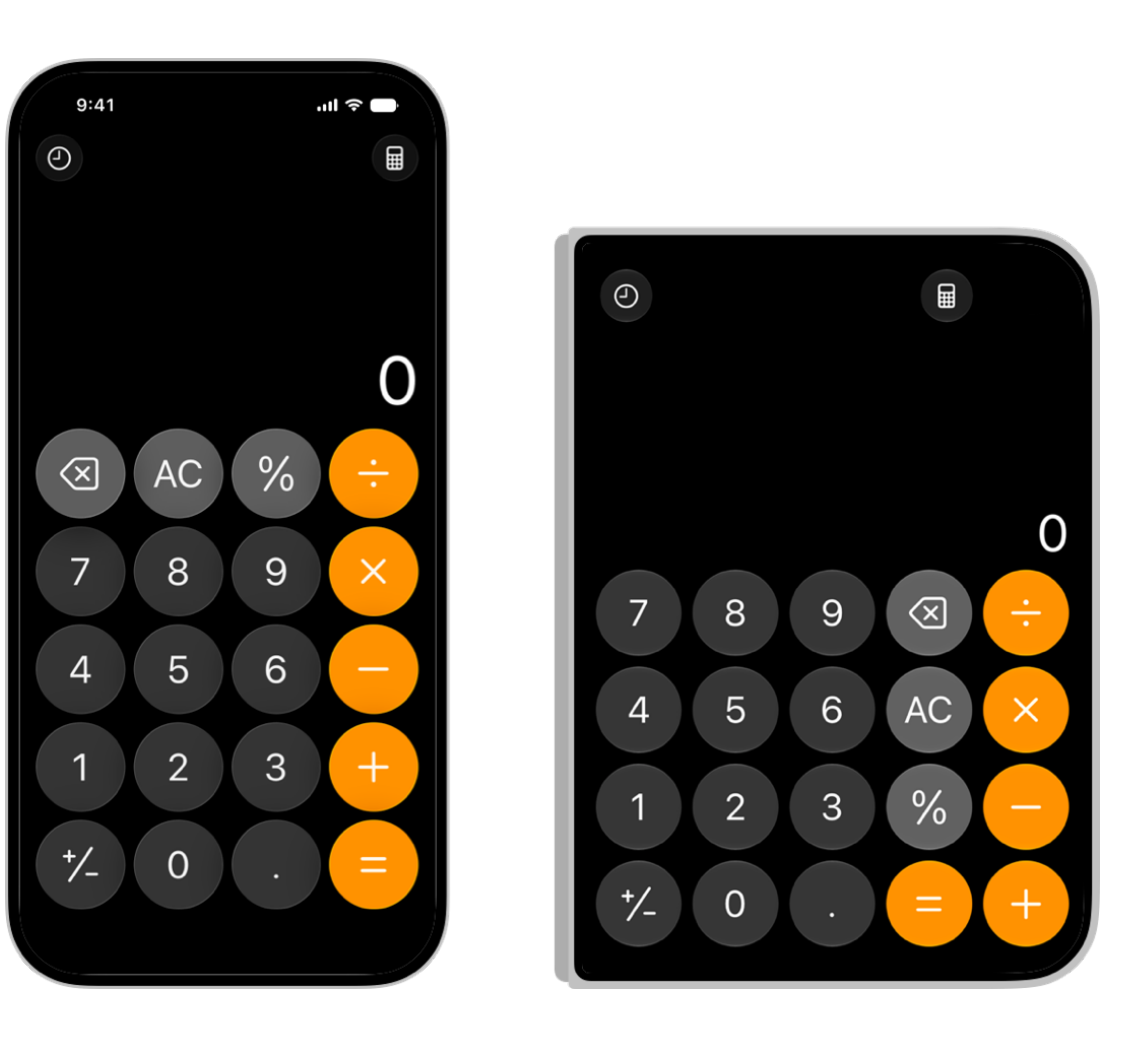
Follow the standard placement order for toolbar items. Reserve the top of the vertical axis for primary navigation controls, like Back or Close, followed by prominent actions, like Done. This preserves familiar navigation patterns while keeping important actions within reach. Keep remaining toolbar items in their original groupings; the system provides a vertical space between items from the top and bottom bars to keep them distinct.

Prioritize frequently used toolbar items to keep them easily available. Items overflow from bottom to top by default. Assign each item a visibility priority to change that order, starting with whole groups and then individual items within a group if you need finer control. For developer guidance, see `ToolbarItemVisibilityPriority` (SwiftUI) or `UIBarButtonItemVisibilityPriority` (UIKit).

Preserve frequently used actions first, like Compose in Mail or New Note in Notes, and keep controls that convey important status, like items with badges, visible longer so people can see them at a glance.

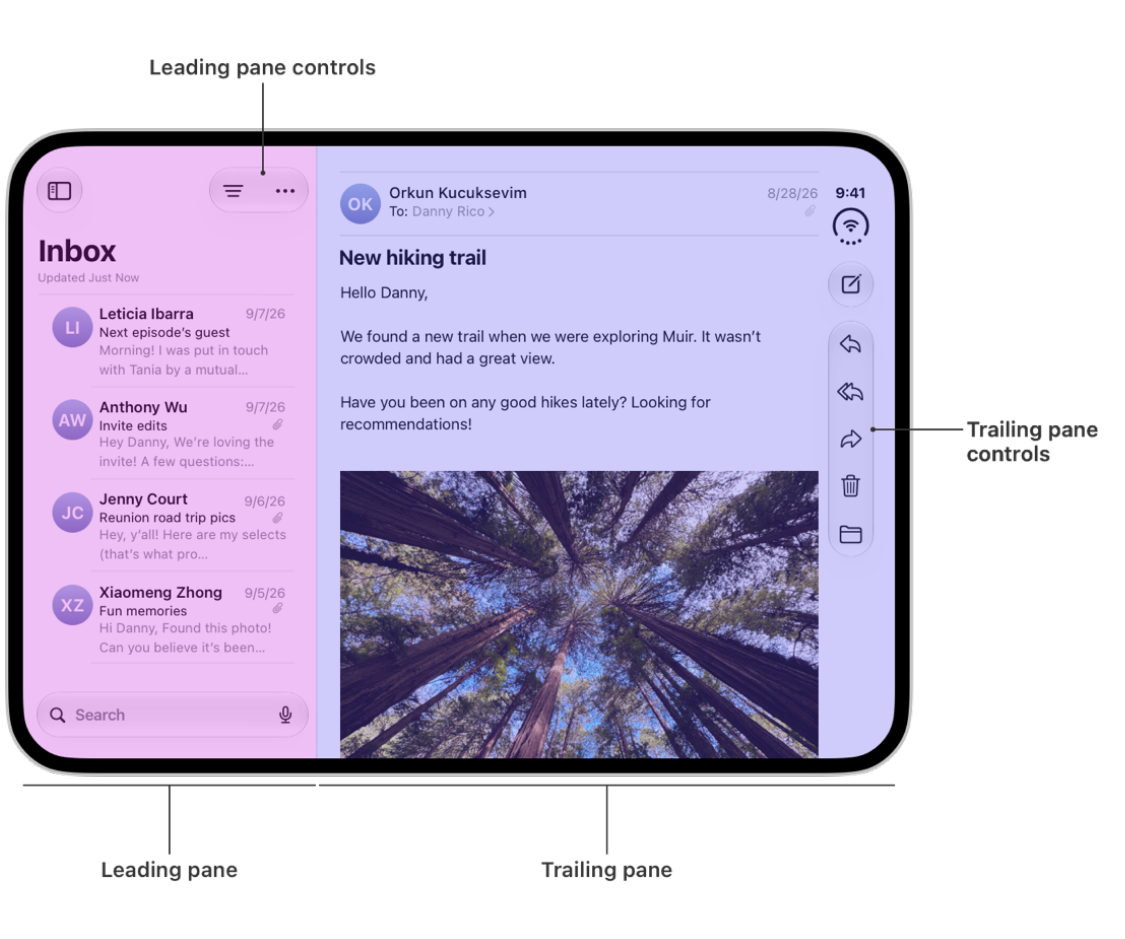
In general, don't override the default bar placement. The position of controls on the vertical axis is one of the core patterns of iPhone Duo. Keeping controls in familiar positions helps people get to know how your app works right away, and reinforces the unified platform experience.

Consider using the full display width for interfaces where bars aren't necessary. Some layouts can span the full display, which works well for visual, immersive interfaces that don't scroll, as long as nothing conflicts with the Dynamic Island or the status bar. Calculator, for example, occupies the full width of the display. You can also combine both approaches, letting a background image or header span the full width while scrollable content stays inset.



Group related toolbar items instead of spacing them manually. Groups you create with `ToolbarItemGroup` (SwiftUI) or `UIBarButtonItemGroup` (UIKit) provide space between items and other groups automatically, and adapt as the available space changes, so avoid adding fixed spacing yourself. For guidance, see Toolbars.

Locate controls near the content they affect. When controls belong to a content area other than the one along the trailing edge, keep them with that area rather than moving them to the side. Proximity makes the relationship between controls and content clear. For example, controls that affect the list of emails in Mail stay above the leading pane to indicate that they apply to the list, rather than the contents of an individual email.



Hello Danny,

We found a new trail when we were exploring Muir. It wasn't crowded and had a great view.

Have you been on any good hikes lately? Looking for recommendations!

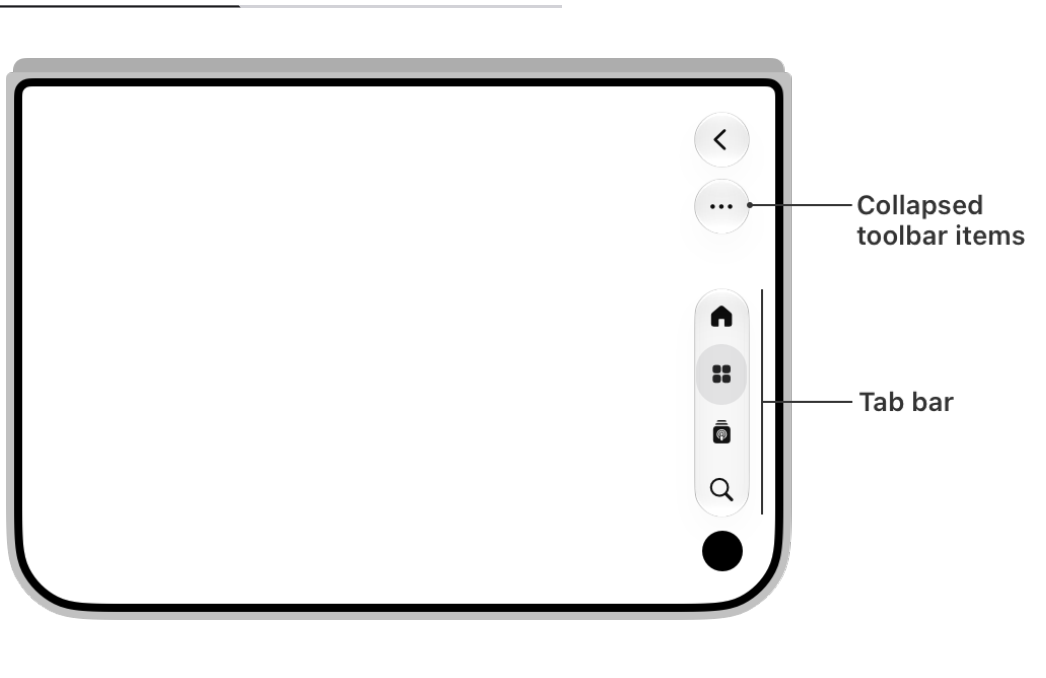
Provide both a title and a symbol for each toolbar item that isn't text-only. Giving both lets the system pick the right representation for the context. Include a title even when an item shows a symbol, because the system uses the title in overflow menus and expanded forms. For developer guidance, see `Label` (SwiftUI) and `UIBarButtonItem` (UIKit).

Keep text-based buttons to a minimum. Labels that include text stay in a horizontal bar, so prefer a symbol wherever one works.

When space is limited, preserve either the toolbar or tab bar based on the experience that the view provides. In navigation-focused experiences, move toolbar items into the overflow menu so the tab bar and primary destinations remain accessible. This is the default bar compression behavior.

In task-oriented experiences, minimize the tab bar to preserve the toolbar actions that are central to completing the task. This mirrors the minimized tab bar behavior present on other iPhone devices. For developer guidance, see `ToolbarVerticalCompressionBehavior` (SwiftUI) and `UIVerticalBarCompressionBehavior` (UIKit).

Toolbar compressed Tab bar compressed



Use the system overflow menu. If your app has its own overflow menu, move those actions into the system menu so people find everything in one place. Reserve the ellipsis symbol for overflow, and give other menus a distinct symbol. For developer guidance, see `ToolbarOverflowMenu` (SwiftUI) and `additionalOverflowItems` (UIKit).

Resources

Related

Apple Design Resources

Screenshot specifications

Designing for iOS

Layout

Developer documentation

Preparing your app for iPhone Duo

`ReservedRegion` — SwiftUI

`UIView.ReservedRegion` — UIKit

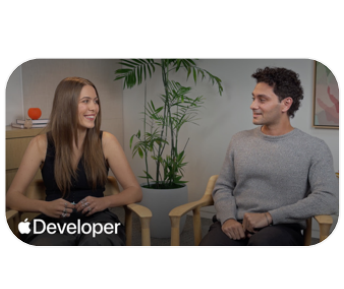
`ArrangementView` — SwiftUI

`UIArrangementViewController` — UIKit

Videos

Design for iPhone Duo

Raise the bar with iPhone Duo



Strike a pose with adaptive layouts on iPhone Duo

Change log

Date	Changes
	New page. Introduces the fundamental

September 9, 2026

concepts of designing for iPhone Duo, including device poses, dynamic layouts across dual displays, and toolbars and tab bars on the vertical axis.

Developer › Documentation

Platforms
iOS
iPadOS
macOS
tvOS
visionOS
watchOS
Tools
Swift
SwiftUI
Swift Playground
TestFlight
Xcode
Xcode Cloud
SF Symbols

Topics & Technologies
Accessibility
Accessories
App Extension
App Store
Audio & Video
Augmented Reality
Design
Distribution
Education
Fonts
Games
Health & Fitness
In-App Purchase
Localization
Maps & Location
Machine Learning & AI
Open Source
Security
Safari & Web

Resources
Documentation
Tutorials
Downloads
Forums
Videos
Support
Support Articles
Contact Us
Bug Reporting
System Status
Account
Apple Developer
App Store Connect
Certificates, IDs, & Profiles
Feedback Assistant

Programs
Apple Developer Program
Apple Developer Enterprise Program
App Store Small Business Program
MFi Program
News Partner Program
Video Partner Program
Security Bounty Program
Security Research Device Program
Events
Meet with Apple
Apple Developer Centers
App Store Awards
Apple Design Awards
Apple Developer Academies
WWDC

To submit feedback on documentation, visit Feedback Assistant.

Light Dark Auto

Copyright © 2026 Apple Inc. All rights reserved.